When objects fall and code runs, A free-falling simulation

IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY
جامعة الإمام عبدالرحمن بن فيصل
كلية العلوم والدراسات الإنسانية – قسم الفيزياء
College of science and Humanities – Physics Department

raydaa M. Mobarki 2240004954@iau.edu.sa
Noof B.Al Binalshaikh 2240002260@iau.edu.sa
Renad I. AlOthayqi 2240002260@iau.edu.sa.
Fatemah J. Alkater. 2240002524@iau.edu.sa
Fatimah H. Alhamad 2240003625@iau.edu.sa

Supervised by: Prof.Ghada I. Ameerah , Ms.Shahad S. Alosaimi

PHYSICS MAGIC FAIR

ABSTRACT:

Free fall is a core concept in physics, describing motion under gravity alone. This experiment models free fall using a Python-based application where users input height and mass, and the system simulates the fall to calculate gravitational acceleration. In traditional labs, diameter and height are measured, and fall time is used with kinematic equations to find acceleration. [1] This project integrates key physics concepts like uniformly accelerated motion, gravity, and Newton's laws, offering students an interactive way to explore the relationships between mass, height, and acceleration.

THEORETICAL BACKGROUND:

When an object falls freely from rest and is subjected to constant acceleration (such as gravitational acceleration), its motion can be described using one-dimensional kinematics equations. since the object starts from rest, the initial velocity equals zero, $v_\circ = 0$, the displacement equation becomes:

$$h = \frac{1}{2}at^2$$

In free-fall motion, the acceleration (a), is equal to the gravitational acceleration (g), therefore the equation becomes:

$$a = g$$

$$h = \frac{1}{2}gt^2$$

From this equation we can isolate and calculate the gravitational acceleration as:

$$g = 2\frac{h}{t^2}$$

The acceleration can be determined using this formula and the measured values of Hight(h) , and time(t) that the object takes to fall. [2]

OBJECTIVES:

- To simulate free-fall motion using python
- Visualizing the relationship between time and displacement.
- Comparing simulated results with theoretical values.

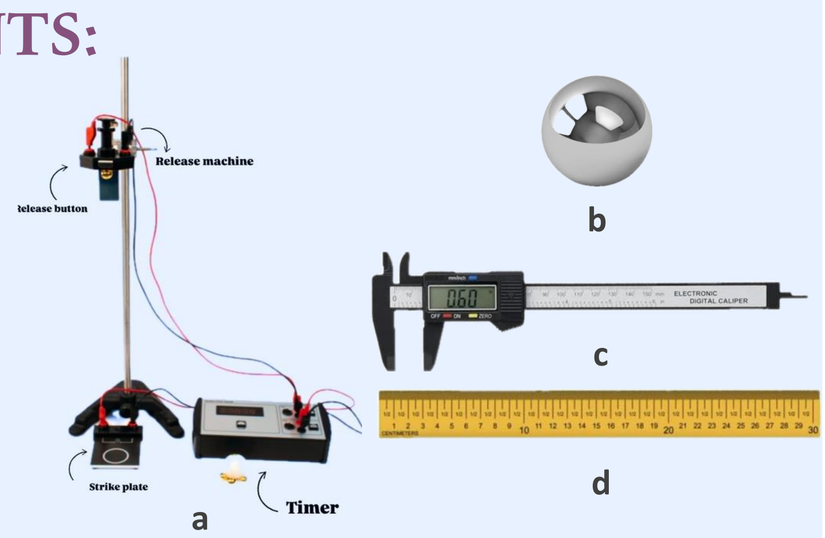
COMPONENTS:

a-
- Release button
- Release machine
- Strike plate
- Timer

b-Steel ball

c-Micrometer caliper

d-Ruler



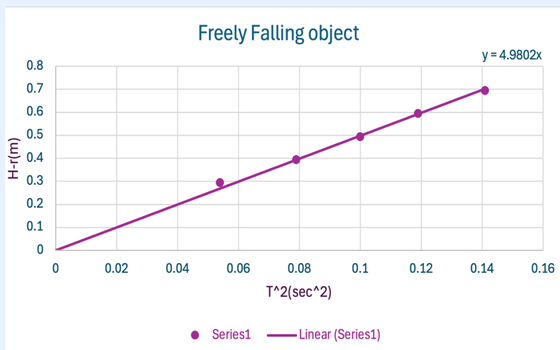
TABLES & GRAPHS:

D(mm)	r(mm)	r(m)
11.95	5.975	0.005975

Table 1

h(cm)	70	60	50	40	30
h(m)	0.7	0.6	0.5	0.4	0.3
H-r(m)	0.694	0.594	0.494	0.394	0.294
T1(msec)	374.45	345.07	316.01	277.88	234.34
T2(msec)	376.64	346.7	316.92	287.74	231.68
T-AVG(mes)	375.54	345.88	316.46	282.81	233.01
T(sec)	0.37554	0.34588	0.31646	0.28281	0.23301
T^2(sec^2)	0.141	0.119	0.1	0.079	0.054

Table 2



The graph shows the relationship between H-r and T^2

DISCUSSION:

The linear fit of the $H - r$ vs T^2 graph gave a slope of approximately $4.98 m/s^2$. According to the equation $h = ½gt^2$, the gravitational acceleration can be calculated as:

$$g = 2 \times slope = 2 \times 4.98 = 9.96\, m/s^2$$

This result is close to the theoretical value of $9.81\, m/s^2$, with a percentage error of about 1.5%. The small difference may be due to minor timing inaccuracies or measurement uncertainties. Overall, the data demonstrates a strong linear relationship, validating the theoretical model of free fall.

REFERENCES:

[1]Serway, R. A., & Jewett, J. W. (2014). Physics for Scientists and Engineers with Modern Physics (9th ed.). Cengage Learning.

[2]Giancoli, D. C. (7th Edition) - Physics: Principles with Applications

Modeling the experiment using Equations:

Input:

1-
```
# calculate the Gravity Acceleration:

import sympy as sy
slope2=sy.symbols("*slope")
print("Gravity Acceleration:")
2 * slope2
```

2-
```
#Free Fall Gravity Equation:

import sympy as sy
h , t =sy.symbols("h t")
g=2*d/t**2
print("Free Fall Gravity Equation:")
g
```

Output:

1- Gravity Acceleration:
$2 * slope$

2- Free Fall Gravity Equation:
$\frac{2d}{t^2}$

Data Tables:

Input:

1-
```
#Creating a Table1 and performing calculations on it

import pandas as pd

D=11.95
rmm=D/2
rm=rmm/1000

data={'D(mm)':[D],'r(mm)':[rmm],'r(m)':[rm]}

df=pd.DataFrame(data)

df=df.style.set_table_styles([{'selector':'thead th','props':[('background-color','teal'),('color', 'white')]}])

df
```

2-
```
#Creating a Table2 and performing calculations on it

import pandas as pd

data={'h(cm)':[70,60,50,40,30],'T1(msec)':[374.45,345.07,316.01,277.88,234.34],'T2(msec)':[376.64,346.7,316.92,287.74,231.68]}

df=pd.DataFrame(data)

df['h(m)']=df['h(cm)']/100
df['H-r(m)']=df['h(m)']-rm
df['T-AVG(mes)']=(df['T1(msec)']+df['T2(msec)'])/ 2
df['T(sec)']=df['T-AVG(mes)']/1000
df['T² (sec²)']=df['T(sec)']**2

order=['h(cm)','h(m)','H-r(m)','T1(msec)','T2(msec)','T-AVG(mes)','T(sec)','T² (sec²)']

df=df[order]

df=df.style.set_table_styles([{'selector':'thead th','props':[('background-color','teal'),('color','white')]}]).format(precision=3)

df
```

Output:

	D(mm)	r(mm)	r(m)
0	11.950000	5.975000	0.005975

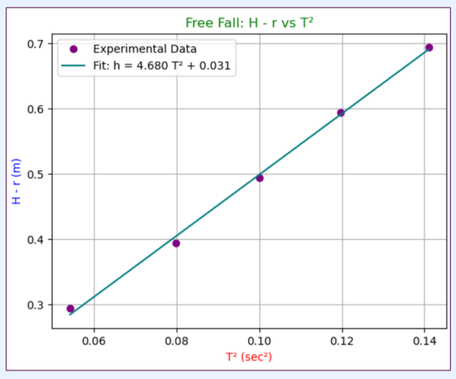
(1)

	h(cm)	h(m)	H-r(m)	T1(msec)	T2(msec)	T-AVG(mes)	T(sec)	T² (sec²)
0	70	0.700	0.694	374.450	376.640	375.545	0.376	0.141
1	60	0.600	0.594	345.070	346.700	345.885	0.346	0.120
2	50	0.500	0.494	316.010	316.920	316.465	0.316	0.100
3	40	0.400	0.394	277.880	287.740	282.810	0.283	0.080
4	30	0.300	0.294	234.340	231.680	233.010	0.233	0.054

(2)

Plot:

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

#extract values from the data frame

x = df.data['T² (sec²)'].values.reshape(-1, 1)
y = df.data['H-r(m)'].values

model = LinearRegression()
model.fit(x, y)

slope = model.coef_[0]
intercept = model.intercept_

plt.plot(x, y, 'o', label="Experimental Data", color="purple")
plt.plot(x, model.predict(x), label=f"Fit: h = {slope:.3f} T² + {intercept:.3f}", color="teal")

plt.xlabel("T² (sec²)", color="red")
plt.ylabel("H - r (m)", color="blue")
plt.title("Free Fall: H - r vs T²", color="green")

plt.legend()
plt.grid(True)
plt.show()
#Slope

import numpy as np

slope = (np.max(y) - np.min(y)) / (np.max(x) - np.min(x))
print("Slope:", slope,'m/s^2')
gravity=2 * slope
print('The gravity acceleration =',round(gravity,2),'m/s^2')
```




```
Slope: 4.611462021098197 m/s^2
The gravity acceleration = 9.22 m/s^2
```

Discussion:

```
# Sorting the data to observe the relationship between T² and displacement
sorted_df = df.sort_values(by='T² (sec²)').reset_index(drop=True)

# Calculating the correlation coefficient between T² and H - r
correlation = sorted_df['T² (sec²)'].corr(sorted_df['H-r(m)'])

# Displaying the sorted data and correlation
print("Sorted Data:")
print(sorted_df)
print(f"\nCorrelation between T² and H - r : {correlation:.4f}")
print ('''

The sorted data and graph both demonstrate a strong linear relationship between
time squared (T²) and displacement (H - r), aligning with the theoretical equation :
h = 0.5 * g * t²


This direct proportionality confirms that the object experiences constant
acceleration due to gravity, which validates the accuracy of our simulation

The correlation coefficient obtained is very close to 1, indicating that
the data closely follows a linear trend as predicted

This supports the objectives of the experiment :
- Simulated free fall motion using Python
- Visualized the relationship between time and displacement
- Compared simulated results with theoretical expectations
''')
```

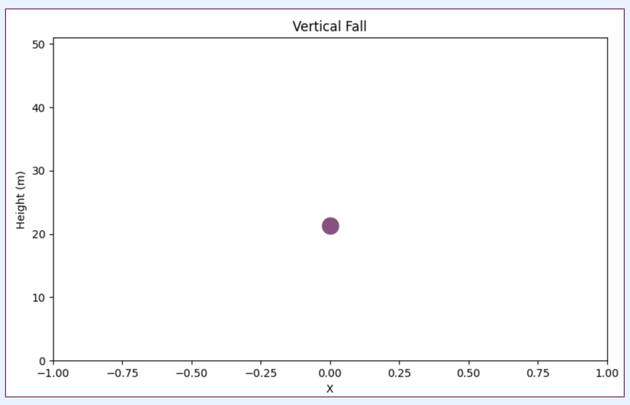
```
Sorted Data:
   T² (sec²)  H-r(m)
0   0.053859  0.1882
1   0.077223  0.2882
2   0.100356  0.3882
3   0.119077  0.4882
4   0.134781  0.5882

Correlation between T² and H - r : 0.9965
```

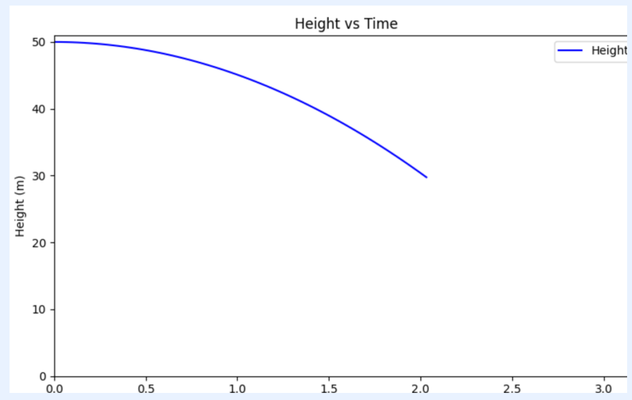
Lab Error:

```
#Accuracy

Ruler=float(input('The smallest value for Ruler:'))
Micrometer=float(input('The smallest value for Micrometer:'))
Timer=float(input('The smallest value for Timer:'))

print('Ruler:',0.5*Ruler,'cm')
print('Micrometer:',0.5*Micrometer,'mm')
print('Timer:',0.5*Timer,'msec')

#Percentage Error:

print('The Percentage Error =' ,abs(((gravity-9.81)/9.81)*100),'%')
```

```
The smallest value for Ruler: 0.1
The smallest value for Micrometer: 0.01
The smallest value for Timer: 0.01
Ruler: 0.05 cm
Micrometer: 0.005 mm
Timer: 0.005 msec
The Percentage Error = 5.984464401667744 %
```

Simulation:





This simulation represents a self-learning experience, where the student utilized various resources and consulted specialists to review and improve the code, which proved to be a valuable opportunity. The simulation was developed using Python, incorporating NumPy for numerical computations, Matplotlib for visual representation, and ipywidgets to enable interactive control. It demonstrates the motion of a falling object, visualizing its height over time in real-time.